ChatGPT

Leveraging Static Graph Visualizations for Collaborative Knowledge Management

Introduction

In an era where graph-based data is increasingly prevalent – from knowledge graphs to risk registers – organizations are seeking better ways to derive insight and share understanding from complex relationships. Recent advances, including the aid of large language models (LLMs) to generate and populate graph data, have led to a surge in graph creation. Yet many teams encounter the *"so what?"* moment: after constructing an elaborate graph, how do we extract and communicate actionable insights? Traditional graph visualization tools offer interactive interfaces to explore nodes and edges, but these interactions are often ephemeral. Changes made by dragging nodes or adjusting a view are lost unless manually captured, making it hard to preserve a particular insight or share a consistent snapshot with others. This whitepaper presents a workflow and methodology that addresses these challenges by using **static graph visualization snapshots** as a core medium for collaboration and iteration. By automatically generating and distributing image-based graph views from live data, teams can maintain a persistent record of their graphs' evolution and make graph analytics more accessible to a broader community.

Challenges in Graph Visualization and Collaboration

Despite the power of graph databases and visualization engines, there are several key challenges in practice:

- **Ephemeral Views:** Interactive graph tools (e.g. web-based explorers) allow users to pan, zoom, filter, and manually rearrange nodes. However, these adjusted views exist only in the moment. As soon as a user navigates away or refreshes, the specific configuration is lost. Important discoveries or layouts can be difficult to reproduce or share with teammates, analogous to performing a manual test without saving the test script.

- **Hairball Effect:** Graphs representing real-world data (organizations, systems, etc.) tend to grow densely connected. Naively visualizing all relationships up to a few degrees of separation from a node often produces an unreadable "big blob" of nodes and edges, sometimes called a hairball graph. Critical details are easily obscured when *everything* is shown at once. Without a strategy to tame this complexity, users may become overwhelmed and fail to extract value, leading to the *"so what?"* dilemma where the graph's purpose isn't clear.

- **Limited Sharing and Recording:** Many graph tools lack easy ways to capture a snapshot of the graph in a format suitable for embedding in documents or messaging platforms. While one could take a manual screenshot, this process is inconvenient and doesn't scale for continuous updates. Moreover, such screenshots are static and detached from the underlying data, making it hard to know when the graph has changed unless one actively rebuilds and recaptures it.

- **Underutilization of Graph Capabilities in Existing Platforms:** There is a gap in leveraging common work platforms as graph data stores. For example, Atlassian JIRA – widely used for

issue tracking – inherently supports issue linking (parent/child, relates-to, etc.), which can form a rich graph of project knowledge. Yet, industry adoption of JIRA as a true graph database has seen little evolution, representing a missed opportunity [1]. The lack of native, user-friendly graph visualization and sharing features in such platforms means organizations aren't fully realizing the value of their relational data.

These challenges hinder the graph community's efforts to make graph data a practical driver of day-to-day decisions. To overcome them, we propose a workflow that treats **graph visualization as a continuously regenerable artifact** – one that can be easily created, shared, and preserved as the graph itself evolves.

Approach Overview: Static Visualization Snapshots from Live Graph Data

Our approach centers on automatically generating **static image snapshots** of graph views directly from the live graph data whenever needed. Rather than relying on interactive GUI manipulation to explore graphs, we programmatically render diagrams that capture specific slices of the graph. This method draws inspiration from the concept of "diagrams as code," where the visual is produced from a textual specification (much like how source code produces software or how test scripts preserve testing steps). In practice, this workflow involves several components working in tandem:

- **Graph Data Source – Single Source of Truth:** The underlying graph data is stored in a system that acts as the authoritative repository. In our experience, we repurposed JIRA as a graph database: each issue represented a node of a certain type, and JIRA's issue link functionality represented typed edges between nodes (e.g., an "Incident" issue *causes* a link to a "Problem" issue, or a "Vulnerability" *is mitigated by* a "Control" issue). JIRA's flexibility with custom issue types and link types made it suitable for modeling complex security and risk data. More generally, any system where relational data can be stored (a graph database like Neo4j, a relational DB, or other issue trackers) could serve this role. The key is that users – including non-technical staff – can update the graph easily via this system's normal interface (forms, tickets, etc.), enabling collaborative graph editing. The data store remains the single source of truth; the visualizations are *derived* from it (and never manually tweaked in isolation).

- **Query & Transformation Layer:** Instead of pulling entire neighborhoods of the graph ad-hoc (which leads to the hairball problem), we define specific *paths or subgraphs of interest* through queries. This can be achieved by writing small scripts or serverless functions that retrieve a subset of the graph relevant to a particular question. For example, one query might retrieve the chain of links from a given incident: *Incident* → *caused by* → *Event* → *exploits* → *Vulnerability* → *has risk* → *Risk* → *part of* → *Risk Register*, and so on, traversing through specific link types. Each step expands the scope by one hop following a particular relationship. By chaining these steps intentionally, we construct a meaningful path that answers a question (such as "What is the full context and impact chain of this security incident?"). The result of the query is then transformed into a diagram definition in a "graph-as-code" format. We have used tools like **PlantUML** and **Mermaid**, which allow us to define nodes and edges in a text markup. This text definition (essentially, code describing the graph view) can be generated on the fly by our script. Importantly, this means the layout or content of the graph visualization is reproducible and versionable – if the underlying data hasn't changed, running the same query will produce the same diagram every time.

- **Automatic Rendering to Image:** The diagram definitions are then rendered by an engine (PlantUML, Mermaid CLI, GraphViz, or other visualization libraries) to produce a static image (e.g., PNG or SVG file). This step can be done by a serverless function, a bot service, or a CI pipeline triggered by certain events (like data updates or on-demand requests). In our implementation, for instance, a serverless function was invoked to generate a PlantUML diagram image whenever a user requested a graph view. The use of automated rendering means no human intervention is needed to lay out the graph or make it presentable – the engine's layout algorithms handle it. We often choose a hierarchical layout (top-down or left-right in PlantUML) to ensure the resulting image is readable and has a logical flow [2] [3]. In some cases, minor tweaks like pinning a root node's position can be done via the diagram language to improve clarity, but these tweaks become part of the code definition and thus persist for future renders.

- **Integration with Collaboration Tools:** The generated graph images are immediately shared in the channels where the team already communicates. In our case, we integrated the system with **Slack** via a chatbot. Team members could use a Slack command or button to request a graph (for example, "/graph incident 1234 full-context"), and the bot would post the resulting image right into the Slack conversation. This made the graph visualization *accessible in context* – during discussions of an incident, the relevant graph snapshot could be pulled up for all participants to see, without requiring anyone to open a separate application. The images could similarly be embedded in documentation, wikis, or attached to JIRA tickets. Because they are images, anyone can view them with no special software. This closes the loop from data to insight: users modify the graph data (e.g., update a ticket), trigger a new visualization, and see the result in the space where they're making decisions. The *entire user journey stays in Slack (or your chosen collaboration hub)*, creating a smooth experience [4] [5].

This approach effectively treats graph visualizations as ephemeral *outputs* that can be generated at will, rather than static diagrams manually drawn in a tool. By automating the creation of these outputs, we ensure that they are always up-to-date and easily reproducible. Each snapshot captures the state of part of the graph at a moment in time, which brings numerous benefits for collaboration and analysis, as discussed next.

Path-Based Visualization: Avoiding the "Hairball"

A cornerstone of this workflow is the concept of **path-based subgraph visualization**. Rather than showing a whole graph or an arbitrary chunk around a node, we define a *path* – a sequence of nodes connected by specific relationships – that represents a logical thread of information. This approach was inspired by the need to avoid the overwhelming complexity that arises if you indiscriminately expand a graph. In practice, the path-based method works as follows:

- We design our graph schema such that every relationship has a clear meaning and, if appropriate, a defined inverse. For example, if an Event node "causes" an Incident node, we also have the inverse relationship "is caused by" from the Incident's perspective. This bi-directional semantic helps in querying: given an Incident, we can easily traverse back to find its cause event, then further to find what vulnerability that event exploited, and so forth. Each edge type can be seen as a verb (and its inverse) connecting two node types.

- Starting from a focal node (or set of nodes) and a particular question, we traverse one relationship at a time in the direction that adds relevant context. Each traversal step filters the graph to only the next layer of interest. For instance, beginning at an *Incident*, a security analyst might ask: "What caused this incident?" This leads to traversing the *causes* relationship to find

the triggering Event. Next they might ask: "What vulnerability did that event exploit?" – traverse the *exploits* link to a Vulnerability node. Then: "What risk does that vulnerability pose?" – traverse *has risk* to a Risk node. "Who is responsible for that risk?" – traverse *has stakeholder* to a Stakeholder (person or team). And so on. At each step, the subgraph grows in a controlled, linear fashion, rather than branching out in all directions. The result is a chain or a tree that is directly tied to a narrative or query.

- After each step (or after completing the full traversal for the query), the current subgraph is rendered to an image. In our workflow, we would generate an updated image every time a new "hop" was added to the path. This incremental visualization has two effects: firstly, it breaks down the complexity into manageable pieces (the early steps yield small graphs, which then expand), and secondly, it provides a **visual history of exploration**. One can literally see the graph view *growing* as new elements are included. If at any point the view becomes confusing or unwieldy, the user can stop or choose a different path, keeping the snapshots for reference.

- The generated diagram for a path often uses a simple linear or hierarchical layout since the data inherently forms a branching structure from the starting node. This yields a cleaner look than a generic force-directed graph of a large neighborhood. For example, using PlantUML's left-to-right layout, the Incident might appear on the left, then an arrow to the Event, then to the Vulnerability, Risk, and Stakeholder to the rightmost side. Such directional layouts make the cause-effect or ownership chains immediately apparent [3].

By focusing on path-specific views, we **avoid the "hairball" effect** where a graph visualization tries to show too much at once. Each image is purposeful: it answers a specific question or illustrates a particular relationship thread. This directly addresses the *"so what?"* factor – instead of showing a tangle of connections that leave viewers wondering what to conclude, each snapshot comes with an implied narrative (often annotated in the Slack message or documentation around it explaining its significance). The path methodology enforces a level of discipline and intentionality when exploring the graph. It is a limitation by design that ironically leads to deeper clarity: users are not tempted to drag in unrelated nodes just because they're connected, but rather to think, *"what's the next relevant step?"*. The static snapshot then crystallizes that chosen perspective for sharing and future reference.

Benefits of Static Snapshot Workflow

Implementing this workflow of on-demand static graph images yielded numerous benefits for our team and can likewise benefit any knowledge graph or graph database initiative:

- **Persistent Visual History:** Each generated image creates a record of the graph's state at a given time or of the particular query made. Teams can store these images (and more importantly, the code/parameters to reproduce them) in version control or knowledge bases. This means as the underlying graph data changes (e.g., new nodes added, relationships updated), new snapshots can be compared with old ones to visually track changes. For example, one snapshot might show an incident's impact graph last week, and a new snapshot shows it today after mitigation actions – simply by comparing the two images, one can see what's been added or removed. This historical capture is invaluable for refactoring and auditing changes. It provides a form of "visual diff" for the graph structure. In our case, having a timeline of risk graph images helped identify when certain risk linkages were introduced or removed, assisting in root-cause analysis of decision changes.

- **Improved Collaboration and Communication:** Graph data, when left in a database or hidden behind a special tool, often fails to reach the people who could benefit from it. By surfacing the graph via images in everyday communication platforms (Slack, email, reports), we make it far more accessible. Non-technical stakeholders, such as business owners or risk managers, don't need to learn a new tool or write queries – they simply view the image that appears in their workflow. This democratizes the graph insight. In our implementation, subject-matter experts who never would have logged into a graph database UI were suddenly engaged, because they could request and discuss graph views through Slack with ease. The static nature of the image also means it can be annotated or highlighted in conversations (e.g., circling a node on the image to draw attention) without worrying that the underlying view will shift. Essentially, the images became a "common language" artifact that everyone from engineers to executives could talk about. As Dinis Cruz noted in his presentation, using such Slack bots and auto-generated diagrams to visualize data empowers data-driven decisions across the organization [6].

- **Faster Iteration and Feedback Loop:** Because generating a new image is as simple as running a query (which we automated via a bot command), the cost of trying out changes in the graph model and seeing their effect is very low. Teams can experiment with adding a new relationship, reorganizing a hierarchy, or correcting a mislinked node, and immediately see the result in a fresh visualization. This tight loop encourages continuous improvement of the data quality. It also makes errors more obvious: if someone mistakenly links a node in the wrong place, the next image might look incorrect or confusing ("Why is this service shown under the wrong department?"), prompting a quick correction. In our risk management use-case, this led to rapid refinement of our data. The visual feedback was more intuitive and immediate than reading raw data tables or text reports. It's analogous to test-driven development in software – quick cycles of making a change and seeing if the outcome matches expectations, except here it's done with graph data and visual output.

- **Focused Insights (Query-Specific Views):** Each static snapshot is generated with a purpose, corresponding to a specific question or scenario. This focus means that the viewers are not left to interpret the data with no guidance; the context is clear. For example, one snapshot might be "Graph of Stakeholders who need to be notified about Incident #1234", another might be "Systems and Applications affected by Vulnerability XYZ". By generating distinct images for each such question, the team creates a library of *visual answers*. This stands in contrast to an interactive graph UI where a user might click around to find answers – here the answer is presented as a coherent picture. This approach aligns with how people make decisions: targeted information at the right time. It also eases incorporation into reports and decision-making documents – instead of a generic graph, a report can include exactly the diagram that supports the specific decision at hand (with a title or caption that matches the question). As evidence of the impact, our use of targeted PlantUML-generated graphs directly supported risk assessment meetings and post-incident reviews by highlighting only the pertinent factors, leading to more fact-based and focused discussions [6].

- **Platform Neutrality and Integration:** The workflow is tool-agnostic in many ways. While we used JIRA, Elastic, PlantUML, and Slack, the concept could be applied with different components (e.g., a Neo4j graph database, a Python script to query and use Graphviz, and Microsoft Teams as the output channel). All that is required is that the pieces connect: data source (graph store) -> query/transform -> diagram generator -> communication channel. This means organizations can leverage existing infrastructure. Notably, even without a dedicated graph database platform, one can still **achieve graph-like capabilities using familiar tools** – our choice of JIRA as a graph store is a testament to this. JIRA's strength in capturing work items and their relationships proved to be an untapped resource for graph-based analysis [1]. By using JIRA in this novel way,

we unlocked additional value without introducing a new system, underscoring the benefit of integrating with current platforms. The same could be said of other issue trackers or wiki systems that allow linking of pages/issues – they can be viewed through this graph lens if one builds the snapshot pipeline.

- **Reproducibility and Documentation:** Because each image comes from a defined query and code template, the process is fully reproducible. Months later, if someone questions why a certain decision was made, the team can re-run the same query on the historical data (if preserved) or at least examine the script that was used. The images in essence document the architecture and relationships in a way that static written documentation often fails to do (or goes outdated). We even embedded these images into markdown documents and generated PDFs as automated reports, creating living documentation that updated whenever the underlying data changed. For example, a weekly report could always pull the latest "Risk register overview" graph image. In this sense, the combination of graph snapshots and automation can serve as a lightweight alternative to maintaining a separate documentation site – the visuals are always in sync with reality, because they are generated from it.

In summary, the static snapshot workflow transformed our use of graph data from a niche technical exercise into a collaborative, continuously evolving practice. It addressed the key pain points of traditional graph visualization by making views persistent, shareable, focused, and easy to regenerate. The next section details how one might implement such a system, highlighting important considerations.

Implementation Considerations

Adopting a static visualization pipeline for graphs does require some planning and engineering effort. Here, we outline the primary considerations and options based on our experience:

- **Data Modeling in the Source:** If using a work-tracking tool (like JIRA) or another non-graph database as your source, invest effort in designing a schema that captures graph semantics cleanly. Define clear issue types (node types) and link types (edge types) that represent your domain (e.g., Incident, Event, Vulnerability, Risk, Control, Asset, etc., with links like *causes, exploits, mitigated-by, part-of,* etc.). Ensure every link type is used consistently and consider defining an inverse for each (either explicitly if the platform allows, or implicitly in your query logic). A well-structured data model will make queries simpler and avoid ambiguity when generating visuals. Dinis Cruz has highlighted the unique value of JIRA in this respect – when configured properly, it can function as an effective graph database for various use cases [1]. The same principle applies to other systems: you want your data layer to accurately represent relationships so the visuals derived from it are meaningful.

- **Synchronization and Query Mechanism:** If the source system is not easily queryable for complex traversals, consider extracting or syncing the data to a query-friendly store. In our case, we pushed all JIRA issue and link data into an **Elasticsearch (ELK) stack** in near-real-time [4]. This allowed our serverless functions to run complex searches (e.g., find all nodes connected via certain link types) quickly, which might have been slower or more cumbersome via the JIRA API directly. Alternatives include using a graph database like Neo4j as an intermediate, or even in-memory graphs. The synchronization could be one-directional (source to query store) and doesn't have to be instantaneous – even syncing every few minutes can be sufficient for interactive use. The query mechanism can be written in any language; we used Python for ease of stringing together queries and calling the PlantUML rendering engine.

- **Diagram Generation Tools:** Choosing the right diagramming engine is important. **PlantUML** was our choice due to its rich support for graph-like diagrams (using UML class or mindmap notations) and the ability to control layouts (e.g., left-to-right). It also can easily be run headlessly (it generates images from command line using GraphViz under the hood). **Mermaid** (which integrates with many markdown tools and wikis) is another good choice for simpler graphs and has a more modern JavaScript-based renderer. **GraphViz (DOT)** is very powerful for complex layouts and could be used directly. In some cases, even JavaScript libraries like D3 or Cytoscape.js could be employed on a server to generate an SVG snapshot of a graph. The considerations include: how easy is it to feed the data into the tool's format, how flexible is the layout, and what image formats are needed. We found that a text-based definition (PlantUML code) stored alongside the data was useful – it acted as an audit log of what was visualized. For instance, we stored the PlantUML text in our Slack bot logs or attached it to the JIRA ticket as an artifact, so one could open it in a PlantUML editor anytime to tweak or view.

- **Automating the Pipeline:** A seamless user experience requires automation at key points. This includes triggering image generation and delivering the output. In Slack, we achieved this with a custom bot that listened for certain commands or even triggers like an issue status change. For other setups, one could use email-based generation (send an email to a particular address and get a reply with an image), a web dashboard where users click options to generate a view, or integration in a wiki (e.g., a macro that renders the latest graph for a given query). The timing of generation can be on-demand (when requested) or scheduled (e.g., nightly refresh of a set of common diagrams). On-demand is more interactive and ensures the latest data usage; scheduled can populate dashboards for routine use. Modern serverless platforms (AWS Lambda, Azure Functions, etc.) are well-suited for executing the render jobs, as they can be invoked via webhooks from Slack or other UIs.

- **Image Distribution and Management:** Once generated, images can be stored on a server or cloud storage and shared via link, or directly uploaded into the communication channel. One should consider retention and indexing of these images. Over time, a large number of snapshots will be produced. It's beneficial to tag them with metadata (timestamp, parameters used, etc.). We indexed our images by key (often the query name and timestamp) in an S3 bucket, which served as a rudimentary library of past graph views. This came in handy when someone recalled "I saw a graph of X last month, can we get that back?" – we could fetch it from the archive by date. If confidentiality is a concern, ensure the images are shared in appropriate private channels or behind authentication as needed, since graph images might contain sensitive organizational information.

- **Layout and Aesthetics:** While the automation handles most of the layout, some attention to aesthetics improves adoption. Simple things like color-coding node types or using icons/logos for certain nodes (if the tool supports it) can make the graph more intelligible at a glance. For example, all *Risk* nodes could be one color, *Incident* nodes another, etc. We used PlantUML's styling capabilities to set different shapes and colors based on node types (defined in our generated code). Additionally, if a graph is too large to be easily legible, consider splitting the query into two smaller queries (thus two images) or increasing the image resolution. Generally, path-based queries kept things small, but occasionally we wanted a "full map" of a subsystem – in those cases we explicitly limited the scope or applied filters (like show only nodes of certain critical types) to avoid visual clutter. The goal is a diagram that can be parsed by a human in a few seconds of viewing. It's a balancing act between completeness and clarity.

- **Maintenance of the System:** Like any pipeline, this one requires some maintenance. As data schemas evolve, the queries and rendering templates may need updating. We built our query

logic to be data-driven (reading configuration of what link types to follow for a given query), so that changes in the schema (e.g. renaming a link type) could be handled with minimal code changes. It's also wise to monitor for failures – e.g., if the diagram generator ever fails (due to bad data leading to an invalid diagram), the system should catch that and perhaps provide an error message rather than silently failing. In practice, we encountered few issues once things were stable, and the users grew to depend on the graph bot as a reliable part of their toolkit.

By considering these aspects, teams can implement a robust solution that fits their environment. The investment pays off by turning raw graph data into a living visual narrative of the organization's knowledge. We illustrate this with a concrete example in the next section.

Case Study: Graph-Based Risk Management Workflow

To ground the discussion, we present a simplified example inspired by a real deployment in an enterprise security context. The organization's goal was to manage risks, incidents, and vulnerabilities in an integrated way, enabling quick understanding of the impact of incidents and proper notification of stakeholders. The data was stored in JIRA with custom issue types and links as described earlier. Here's how the static visualization workflow was applied:

1. Schema Design: The team defined custom issue types in JIRA: **Incident**, **Event**, **Vulnerability**, **Risk**, **Stakeholder**, **Asset**, **Application**, **Business Unit**, etc. They also created custom link types to represent relationships: *Event causes Incident*, *Event exploits Vulnerability*, *Vulnerability has Risk*, *Risk impacts Asset*, *Asset is part of Application*, *Application belongs to Business Unit*, *Stakeholder owns Risk*, among others. Every relationship was given a clear direction and meaning, effectively creating an ontology of how these entities connect. This structured the underlying graph of thousands of issues in JIRA.

2. Triggering an Analysis: When a new Incident was logged, the security team's workflow (via the Slack bot) would automatically generate an "Incident Context" graph. The bot would take the Incident ID and run a predefined query path: - Start at the Incident node. - Traverse the *caused by* link to find the Event that caused it (if any). - Traverse *exploits* from that Event to find the Vulnerability exploited. - Traverse *has risk* from the Vulnerability to find the associated Risk entry. - Traverse *owns* (inverse of *has risk*) to find which Stakeholder (person or team) is responsible for that risk. - Traverse *impacts* from the Risk to see if any Asset or Application is directly affected. - Traverse *belongs to* from an Application to identify the Business Unit or Department.

Each of these steps added one "layer" to the graph. The final result was a chain linking: **Business Unit – Application – Asset – Risk – Vulnerability – Event – Incident – Stakeholder** (somewhat simplified for this example). The bot generated a PlantUML diagram from this data, enforcing a left-to-right layout starting from the Business Unit down to the Incident and then the Stakeholder. It then posted the resulting image to the Slack channel of the incident response team.

3. Visualization Example: The image in Slack showed a clear story. On the left, a node representing the *Business Unit* (say, "Payments Division") connected to the *Application* ("Payment Gateway Service") that was affected. The Application node connected to the *Asset* (perhaps a server or cluster name) that was impacted. That Asset node linked to the *Risk* ("Loss of PII data risk") that had been identified for that asset. The Risk node linked to the *Vulnerability* (e.g., "Open S3 bucket vulnerability") that was exploited. The Vulnerability node linked to the *Event* ("Data exfiltration attack on Jan 5") that caused it to trigger. Finally, the Event node linked to the *Incident* ("Incident 1234: Data Breach") on the far right. Separately, the *Risk* node also had a link to a *Stakeholder* ("John Doe – CISO") who was the owner of that risk. The layout was arranged so that this Stakeholder appeared above the Risk node for clarity. All nodes were

color-coded by type (Incidents in red, Risks in orange, etc.), and the links were labeled with the relationship verbs (e.g., “exploits”, “causes”, “owns”) to make the semantics clear. Although this graph had nearly eight different entity types, it was presented in an easy-to-read linear format without extraneous connections – one could follow the chain of events and accountability at a glance.

4. Actionable Insights: With this visualization, the incident response team could immediately discuss relevant questions: - **Scope of Impact:** Which application and business unit are affected? (From the left side of the chain – Payments Division’s Payment Gateway). - **Responsible Parties:** Who needs to be informed or involved? (Stakeholder John Doe, the owner of the risk, plus likely the management of the Payments Division). - **Risk Context:** Was this risk known, and what was its status? (The Risk node could include an attribute like status “Accepted” or “Mitigated” – if it was accepted, this incident might trigger reconsideration). - **Root Cause:** What vulnerability led to this, and was it known? (The presence of a known Vulnerability node suggests it was perhaps identified but not fixed, which is a crucial discussion point). - **Preventive Measures:** Is there a control or mitigation in place for this risk? (If the graph had a “Control” node linked to the Risk, it could show up as well – in this example, suppose a Control existed but failed, that could be another branch).

Because the graph was generated automatically, it was always up to date. If during the response someone added a new link (maybe they discover an additional affected asset and link it to the Incident), the bot could be asked to regenerate the image, and the new asset would appear on the diagram, ensuring everyone has the latest picture.

5. Outcome and Follow-up: The static images were not only used in real-time, but also pasted into the post-incident report. They served as documentation of “what happened” in terms of cause and effect. Later, when performing a “lessons learned” review, the team pulled up the image to walk through the timeline again. In fact, six months later, when auditing incident trends, the team had a collection of these diagrams to look at patterns (for instance, noticing that a particular Business Unit showed up frequently at the leftmost side, indicating perhaps a systemic issue in that division). The practice of generating and saving these graph snapshots turned into an informal knowledge base for the organization’s security posture.

This case study illustrates how the general principles of our approach can be applied in a concrete scenario. The key was the combination of **structured data, automated generation, and in-context sharing**. The graph community can adopt similar strategies in many other domains – from IT architecture mapping, to fraud investigation link analysis, to social network analytics – anywhere that relationships exist and need to be understood collaboratively.

Future Directions and Conclusion

The experience of using static graph visualization snapshots has shown significant advantages in making graph data usable and valuable to a broad audience. However, there is ample room for further innovation, and the community is just beginning to tap into it. We note a few future directions and concluding thoughts:

- **Enhanced Tool Support:** We envision mainstream graph and work-tracking tools incorporating features to natively snapshot and share graph views. For example, JIRA (as powerful as it is as a graph store) could provide a one-click way to generate a diagram of selected issues and their links. As Dinis Cruz observed, more effort by platform vendors to leverage their graph capabilities would greatly benefit users [1]. The current workflow relies on custom scripts and bots, which, while effective, require technical setup. If such capabilities were built-in, more teams

could adopt graph thinking without a heavy engineering investment. The onus is on both tool makers and the graph community to push for this integration.

- **Interactivity on Demand:** While static images are great for stability and record-keeping, there might be hybrid approaches where an image can be converted into an interactive view when needed. For instance, a snapshot could include metadata that allows a user to click a "Open in Graph Explorer" button to load that exact subgraph in an interactive tool. This would combine the best of both worlds: static for communication, interactive for deeper dives. Some research prototypes already capture static visualizations with embedded graph data for later exploration [7] [8], hinting at possibilities like reversible or layered visualizations.

- **LLM-Assisted Graph Generation:** With large language models becoming more integrated into workflows, we foresee LLMs assisting in this domain too – not only in populating graph data from unstructured sources, but also in generating the queries or diagram definitions for snapshots. One could imagine asking a chatbot, "Show me the impact graph of Incident 1234," and under the hood the LLM understands the schema and produces the appropriate path query and visualization code. This could simplify the user interface even further – users describe what they want in natural language, and the system produces the graph image with no need for predefined commands.

- **Standardization of "Graph as Code":** Just as infrastructure as code and configuration as code have revolutionized DevOps, a standardized way to represent graph visuals as code could emerge. PlantUML and DOT are early examples but are somewhat limited to tech-savvy users. A higher-level, user-friendly descriptive language for graph snapshots (perhaps JSON or YAML based, or even a visual DSL) could make it easier to save and exchange graph views. This would also facilitate better version control and diffing of graphs, enabling more formal change management on knowledge graphs.

- **Broadening Use Cases:** While our focus was on risk and incident management, the approach is generic. Communities focusing on social graphs, knowledge management, project management, etc., can all benefit from delivering their graph insights in bite-sized, purposeful images. Sharing this workflow idea widely (through whitepapers like this) can spark new applications. For instance, a marketing team might use it to visualize customer journey connections, or a research team might map out citation networks in a static shareable form. The more it's tried in different contexts, the more patterns and best practices will emerge.

In conclusion, transforming ephemeral graph explorations into persistent, communicable snapshots addresses a critical gap in current graph technology usage. By **combining automatic graph rendering with collaborative tools**, we enable a feedback loop where graphs are not just built and forgotten, but continuously consulted and refined. The professional graph community is encouraged to consider "graph screenshot" workflows as part of their toolkit – not as a step backward to static media, but as a strategy to *propel graph adoption forward* through clarity and shareability. As our experience demonstrates, this approach empowered teams to make fact-based, graph-informed decisions in real time [6], and helped non-experts actively participate in maintaining and using graph data. Embracing these principles can lead to a future where graph-driven insights are truly at the fingertips of everyone in an organization, fulfilling the promise that **"enabling the business to think in graphs"** is not just for specialists but for all [9].

References and Sources

- Cruz, Dinis. *Creating a Graph-Based Security Organisation.* OWASP London Chapter, Apr 2019. (Slides) – Describes the use of JIRA as a graph database, Slack bots, and PlantUML diagrams to visualize and interact with security graph data [6] [4]. This presentation highlights how graph-driven workflows enable data-driven risk management decisions.
- Cruz, Dinis. "Jira as Graph DB" – LinkedIn post, 2023. – Discusses the concept of using Atlassian JIRA to store graph nodes and edges via issues and links, noting that JIRA's potential as a graph database is underutilized [1]. Emphasizes the value of leveraging existing tools for graph data and calls for more innovation in this space.
- **PlantUML Documentation** – *Diagrams as Code* approach for generating UML and other diagrams from text scripts. Useful for understanding how static diagram generation can be automated in workflows (relevant to the method described in this paper).
- **Mermaid.js Documentation** – Another diagrams-as-code tool often used in wikis and docs for creating quick graphs and visualizations from text definitions. Illustrates the growing trend of embedding auto-generated diagrams in documentation platforms.
- UCSC OSPO (2023). *Static and Interactive Visualization Capture* – Explores methods to capture static visualizations with embedded metadata for later interaction [7]. Relevant as a glimpse into how static and interactive approaches can be merged in the future. (Available at UCSC Open Source Program Office blog).

---

[1] Jira as Graph DB | Dinis Cruz | 10 comments
https://www.linkedin.com/posts/diniscruz_jira-as-graph-db-activity-7293099257547354112-z4xs

[2] [3] [4] [5] [6] [9] Creating a graph based security organisation - Apr 2019 (OWASP London chapter meeting) | PPT
https://www.slideshare.net/slideshow/creating-a-graph-based-security-organisation-apr-2019-owasp-london-chapter-meeting/140239010

[7] Static and Interactive Visualization Capture - UCSC OSPO
https://ucsc-ospo.github.io/report/osre24/niu/repro-vis/20250301-aryas/

[8] Static visualizations do not exist | by Dominikus Baur - Prototypr
https://blog.prototypr.io/static-visualizations-do-not-exist-b2b8de1ed224